

White Rabbit PTP Core User's Manual

December 2013 (wrpc-v2.1-dirty)
Building and Running

Grzegorz Daniluk (CERN BE-CO-HT)

Table of Contents

Introduction	1
1 Software and hardware requirements	1
1.1 Repositories and Releases.....	1
1.2 Hardware needed	1
2 Building the Core	2
2.1 HDL synthesis.....	2
2.2 LM32 software compilation	3
3 Running and Configuring	4
3.1 Downloading firmware to SPEC	4
3.2 Writing EEPROM and calibration	5
3.3 Running the Core	6
4 Troubleshooting	8
5 Questions, reporting bugs.....	9
Appendix A WRPC Shell Commands.....	10
Appendix B WRPC GUI elements.....	12

Introduction

This is the user manual for the White Rabbit PTP Core developed on [ohwr.org](http://www.ohwr.org). It describes the building and running process. If you don't want to get your hands dirty and prefer to use the binaries available at <http://www.ohwr.org/projects/wr-cores/files> please skip [Chapter 2 \[Building the Core\]](#), [page 2](#) and move forward directly to [Chapter 3 \[Running and Configuring\]](#), [page 4](#).

1 Software and hardware requirements

1.1 Repositories and Releases

This manual is about the official wrpc-v2.1-dirty stable release of White Rabbit PTP Core (WRPC).

The code and documentation for the project is distributed in the following places:

<http://www.ohwr.org/projects/wr-cores/documents>

This place hosts the pdf documentation for every official release.

<http://www.ohwr.org/projects/wr-cores/files>

Here we place the *.tar.gz* file for every release, including the *git* tree and synthesized/compiled binaries

<git://ohwr.org/hdl-core-lib/wr-cores.git>

Read-only repository with the complete HDL sources of WRPC

<git://ohwr.org/hdl-core-lib/wr-cores/wrpc-sw.git>

Read-only repository with the WRPC LM32 software (incl. WR PTP daemon)

Other tools useful for building and running WRPC can be downloaded from the following locations:

<http://www.ohwr.org/projects/hdl-make/files>

hdlmake is used in the HDL synthesis process to create a Makefile based on the set of Manifest files.

<http://www.ohwr.org/attachments/download/1133/lm32.tar.xz>

LM32 toolchain is used to compile the WRPC firmware (software). This specific file is linked from the *files* tab of the *wrpc-sw* project.

The repositories containing the WRPC gateware and software (*wr-cores*, *wrpc-sw*) are tagged with *wrpc-v2.1* tag. Other tools used to build the core and load it into SPEC board should be used in their newest stable release.

Any official hot fixes, if any, for this release will live in the branch called *wrpc-v2.1-fixes*, in each WRPC repository.

1.2 Hardware needed

The absolutely minimum hardware you need to build and run the White Rabbit PTP Core is a PC computer with Linux and one Simple PCIe FMC Carrier (SPEC) - <http://www.ohwr.org/projects/spec>. However, it is highly recommended to use also the DIO FMC card (<http://www.ohwr.org/projects/fmc-dio-5chttla>) for storing calibration values and configuration in EEPROM(described later) as well as feeding 1-PPS and 10MHz from external clock and outputting 1-PPS aligned to WR time. To test the White Rabbit synchronization, you will also need:

- second SPEC board with DIO FMC or a White Rabbit Switch;
- pair of WR-supported SFP transceivers (the list of supported SFPs can be found on our wiki page <http://www.ohwr.org/projects/white-rabbit/wiki/SFP>)
- a roll of G652, single mode fiber to connect your SPECS or SPEC with WR Switch.

2 Building the Core

Note: you can skip this chapter if you want to use the release binaries available for download from *ohwr.org*.

Building the White Rabbit PTP Core is a two step process. First you have to synthesize the FPGA firmware (gateway) and then compile the software which will be running on a softcore LM32 CPU instantiated inside the gateway. Optionally, you can skip the compilation of LM32 software, since the synthesized gateway contains default LM32 software.

To perform the steps below you will need a computer running Linux.

2.1 HDL synthesis

Before running the synthesis process you have to make sure that your environment is set up correctly. You need the Xilinx ISE software with at least free of charge WebPack license. *ISE* provides a set of scripts: *settings32.sh*, *settings32.csh*, *settings64.sh* and *settings64.csh* that configure all the system variables required by the Xilinx software. Depending on the shell you use and whether your Linux is 32 or 64-bits you should execute one of them before other tools are used. For 32-bit system and BASH shell you should call:

```
/opt/Xilinx/<version>/ISE_DS/settings32.sh
```

The easiest way to ensure that *ISE*-related variables are always set in your shell is adding the execution of the script above to your *bash.rc* file. You can check if the shell is configured correctly by verifying if *\$XILINX* variable contains the path to your *ISE* installation directory.

Note: current version of *hdlmake* tool developed at CERN requires modification of *\$XILINX* variable after *settings* script execution. This (provided that the installation path for *ISE* is */opt/Xilinx/<version>*) should be the following:

```
$ export XILINX=/opt/Xilinx/<version>/ISE_DS
```

Note: the Xilinx project file included in the WRPC sources was created with Xilinx ISE 14.5. It is however recommended to use the newest available version of ISE software.

HDL sources for WR PTP Core could be synthesized using Xilinx ISE without any additional tools, but using *hdlmake* is more convenient. It creates a synthesis Makefile and ISE project file based on a set of Manifest.py files deployed among the directories inside the *wr-cores* repository. First, please download the *hdlmake* binary from its location given in [Section 1.1 \[Repositories and Releases\]](#), page 1 and make it executable. At the time this document is written, the most recent stable version of *hdlmake* is 1.0:

```
$ wget http://www.ohwr.org/attachments/download/2070/hdlmake-v1.0
$ chmod a+x hdlmake-v1.0
```

It is recommended to add the *hdlmake* binary location to your *\$PATH* environment variable to be able to call it from any directory:

```
$ export PATH=<your_hdlmake_location>:$PATH
```

Note: the *hdlmake* usage instructions here are based on version 1.0. If you use more recent release or a development version, please be aware that its execution parameters may change. In that case please refer to *hdlmake* documentation.

Having Xilinx ISE software and *hdlmake* in place, you can clone the main WR PTP CORE git repository and start building the FPGA bitstream. First, please create a local copy of the *wr-cores* in the preferred location in your system.

```
$ git clone git://ohwr.org/hdl-core-lib/wr-cores.git <your_location>/wr-cores
$ cd <your_location>/wr-cores
```

To build the gateway using sources of a stable release wrpc-v2.1, you have to checkout the proper git tag:

```
$ git checkout wrpc-v2.1
```

If you use *wr-cores* within another project (like *wr-nic*), you may need to check out another release tag for this repository. Please refer to the project's documentation to find out which version of this package you need to build.

You also need to fetch other git repositories containing modules essential for WR PTP CORE. They are configured as git submodules inside the *wr-cores* repository:

```
$ git submodule init
$ git submodule update
```

The local copies of the submodules are stored to:

```
<your_location>/wr-cores/ip_cores
```

The subdirectory which contains the main synthesis Manifest.py for SPEC board and in which you should perform the whole process is:

```
$ cd <your_location>/wr-cores/syn/spec_1_1/wr_core_demo/
```

First, please call *hdlmake* to create synthesis Makefile for Xilinx ISE:

```
$ hdlmake-v1.0 --make-ise --ise-proj
```

After that, the actual synthesis is just the matter of executing:

```
$ make
```

This takes (depending on your computer speed) about 15 minutes and should create two files with FPGA firmware: *spec_top.bit* and *spec_top.bin*. The former can be downloaded to FPGA with Xilinx Platform Cable using e.g. *Xilinx Impact*. The latter can be used with the kernel drivers from *spec-sw* repository (check example in [Chapter 3 \[Running and Configuring\]](#), page 4).

If, on the other hand, you would like to clean-up the repository and rebuild everything from scratch you can use the following commands:

- `$ make clean` - removes all synthesis reports and log files;
- `$ make mrproper` - removes *spec_top.bin* and *spec_top.bit* files;

2.2 LM32 software compilation

Note: By default, the release LM32 software is embedded inside the FPGA bitstream you've downloaded from *ohwr.org* or synthesized in the previous chapter. That means you don't have to do manual compilation of LM32 software unless you want to use a development version or you've made some changes required by your application.

To compile the LM32 software for White Rabbit PTP Core you will need to download and unpack the LM32 toolchain from the location mentioned in [Section 1.1 \[Repositories and Releases\]](#), page 1:

```
$ wget http://www.ohwr.org/attachments/download/1133/lm32.tar.xz
$ tar xJf lm32.tar.xz -C <your_lm32_location>
```

Then you need to set the `CROSS_COMPILE` variable in order to compile the software for a LM32 processor:

```
$ export CROSS_COMPILE="<your_lm32_location>/lm32/bin/lm32-elf-"
```

To get the sources of WRPC software please clone the *wrpc-sw* git repository tagged with *wrpc-v2.1* tag. If you use *wrpc* within another project, you may need to checkout a different tag or a specific commit; if this applies, please refer to the documentation of the other package to find the exact version you need to reproduce the released binaries before you make your changes.

```
$ git clone git://ohwr.org/hdl-core-lib/wr-cores/wrpc-sw.git <your_location>/wrpc-sw
$ cd <your_location>/wrpc-sw
$ git checkout wrpc-v2.1 # or "git checkout master"
```

Before you can compile *wrpc-sw* you need to make a few configuration choices. The package uses *Kconfig* as a configuration engine, so you may run one of the following commands (the first is text-mode, the second uses a KDE GUI and the third uses a Gnome GUI):

```
$ make menuconfig
$ make xconfig
$ make gconfig
```

Other *Kconfig* target applies, like *config*, *oldconfig* and so on. A few default known-good configurations are found in *./configs* and you choose one by *makeing* it by name:

```
$ make spec_defconfig
```

The most important configuration choice at this point in time is whether to enable Etherbone or not. It is disabled by default in *spec_defconfig* and enabled by default in *etherbone_defconfig*.

After the package is configured, just run *make* without parameters to build your binary file:

```
$ make
```

The first time you build, the *Makefile* automatically downloads the *git submodules* of this package, unless you already did that by hand. The second and later build won't download anything from the network.

The resulting binary *wrc.bin* can be then used with the loader from *spec-sw* software package to program the LM32 inside the White Rabbit PTP Core ([Chapter 3 \[Running and Configuring\]](#), [page 4](#)).

3 Running and Configuring

3.1 Downloading firmware to SPEC

There is a SPEC board software support(SPEC-SW) project in *ohwr.org*. It contains a set of Linux kernel drivers and user space tools, written by Alessandro Rubini and Tomasz Wlostowski, that interact with a SPEC board plugged into a PCI-Express slot.

Instructions in this section are based on the release *2013-05* of SPEC-SW and are limited to absolute minimum required to load WRPC FPGA and LM32 firmware. Full manual for SPEC-SW can be found at:

<http://www.ohwr.org/attachments/download/2134/spec-sw-2013-05-release.pdf>

If there is a more recent version of the SPEC software support, the up-to-date documentation can always be found in *doc/* subdirectory of SPEC-SW git repository.

First, please clone the git repository of SPEC-SW package and build it:

```
$ git clone git://ohwr.org/fmc-projects/spec/spec-sw.git <your_specsw_location>
$ cd <your_specsw_location>
$ git checkout spec-sw-v2013-05
$ make
```

Then you have to copy your *spec_top.bin* generated in [Section 2.1 \[HDL synthesis\]](#), [page 2](#) or downloaded from *ohwr.org* to */lib/firmware/fmc/*. changing its name:

Note: the commands below have to be executed with superuser rights

```
$ sudo cp <your_location>/wr-cores/syn/spec_1_1/wr_core_demo/spec_top.bin \
    /lib/firmware/fmc/spec-demo.bin
```

You have to download also the "golden" firmware for SPEC card. It is used by the drivers to recognize the hardware:

```
$ wget http://www.ohwr.org/attachments/download/1756/spec-init.bin-2012-12-14
$ sudo mv spec-init.bin-2012-12-14 /lib/firmware/fmc/spec-init.bin
```

Now, you are ready to load necessary drivers that configure the Spartan 6 FPGA with the given bitstream (make sure you are in *<your_specs_location>*):

```
$ sudo insmod fmc-bus/kernel/fmc.ko
$ sudo insmod kernel/spec.ko
$ sudo insmod fmc-bus/kernel/fmc-trivial.ko gateway=fmc/spec-demo.bin
```

To check if the FPGA firmware file was found by the driver and correctly loaded to FPGA the *dmesg* Linux command can be called. Among plenty of messages you should be able to find something very similar to:

```
[1639675.431979] spec 0000:0b:00.0: probe for device 000b:0000
[1639675.431992] spec 0000:0b:00.0: PCI INT A -> GSI 16 (level, low) -> IRQ 16
[1639675.435246] spec 0000:0b:00.0: got file "fmc/spec-init.bin", 1484404 (0x16a674) bytes
[1639675.625773] spec 0000:0b:00.0: FPGA programming successful
[1639675.994110] spec 0000:0b:00.0: mezzanine 0
[1639675.994111]          EEPROM has no FRU information
[1639705.910703] fmc fmc-0b00: Driver has no ID: matches all
[1639705.910731] spec 0000:0b:00.0: reprogramming with fmc/spec-demo.bin
[1639706.104417] spec 0000:0b:00.0: FPGA programming successful
```

If everything went right up to this moment you have your board running the FPGA bitstream with default LM32 software. If you want to load your own *wrc.bin* built from *wrpc-sw* repository you can use the *spec-cl* tool. Programming is done with the simple command below:

```
$ sudo tools/spec-cl <your_location>/wrpc-sw/wrc.bin
```

Now you should be able to start the Virtual-UART tool (also part of the SPEC-SW package) that will be used to interact with the White Rabbit PTP Core Shell:

```
$ sudo tools/spec-vuart
```

If you are able to see the WRPC Shell prompt *wrc#* that means the Core is up and running on your SPEC. Congratulations !

3.2 Writing EEPROM and calibration

You should perform few configuration steps through the WRPC shell before you start testing and using the firmware.

Note: the examples below describe only a subset of WRPC Shell commands required to make a basic configuration and calibration. A full description of all supported commands can be found in [Appendix A \[WRPC Shell commands\]](#), [page 10](#).

Before making the configuration changes, it is good to stop the PTP daemon. Then, the debug messages from daemon would not show up to the console while you will interact with the shell.

```
wrc# ptp stop
```

If your SPEC has any Mezzanine board plugged into the FMC connector (e.g. DIO, Fine Delay, TDC...) then you can create a calibration database inside the FMC EEPROM. The example

below presents WRPC Shell commands which create an empty SFP database and add two Axcen transceivers with deltaTx, deltaRx and alpha parameters associated with them.

```
wrc# sfp erase
wrc# sfp add AXGE-1254-0531 180707 148323 72169888
wrc# sfp add AXGE-3454-0531 180707 148323 -73685416
```

To check the content of the SFP database you can execute the *sfp show* shell command.

Note: The deltaTx and deltaRx parameters above are the default ones for *wrpc-2.1* release bitstream available on *ohwr.org* running on SPEC v4 board and calibrated to the port 1 of WR Switch v3.3. Those values as well as the parameters for the WR Switch are available on the calibration wiki page (<http://www.ohwr.org/projects/white-rabbit/wiki/Calibration>). However, if you re-synthesize the firmware or want to have most accurate estimation of fixed delays and alpha of your fiber you should read and perform the WR Calibration procedure (<http://www.ohwr.org/documents/213>).

The WR PTP Core's mode of operation (GrandMaster/Master/Slave) can be set using the *mode* shell command:

```
wrc# mode gm          # for GrandMaster mode
wrc# mode master      # for Master mode
wrc# mode slave       # for Slave mode
```

This stops the PTP daemon, changes the mode of operation, but does not start it back automatically. Therefore after changing it you need to start the daemon manually:

```
wrc# ptp start
```

Note: For running in GrandMaster mode, you need to provide 1-PPS and 10MHz signal from external source (e.g. GPS receiver or Cesium clock). Please connect then 1-PPS signal to LEMO No.4 and 10MHz to LEMO No.5 connector of FMC DIO board.

One option is to type all those commands to initialize the WRPC software to the required state every time the Core starts. However, you can also write your own init script to the EEPROM and WRPC software will execute it each time it starts (also when coming back from reset after programming the LM32). A simple script that reads SFP parameters from the EEPROM, configures the WR mode to Slave and starts the PTP daemon is presented below:

```
wrc# init erase
wrc# init add ptp stop
wrc# init add sfp detect
wrc# init add sfp match
wrc# init add mode slave
wrc# init add calibration
wrc# init add ptp start
```

Almost exactly the same one can be used for running WRPC in GrandMaster or Master mode. The only difference would be *init add mode slave* vs. *init add mode gm* or *init add mode master*.

3.3 Running the Core

Having the SFP database, and the init script created in [Section 3.2 \[Writing EEPROM and calibration\]](#), [page 5](#) you can restart the WR PTP Core by reprogramming the LM32 software (with *spec-cl* tool) or by typing the shell command:

```
wrc# init boot
```

You should see the log messages that confirm the init script execution:


```
(...)  
WR Core: starting up...  
W1: f80000036c38ee28  
get_persistent_mac: Using W1 serial number  
ID: cafebabe  
t24p read from EEPROM: 6900 ps  
Loops per jiffy: 20820  
Locking PLL  
executing: ptp stop  
executing: sfp detect  
AXGE-1254-0531  
executing: sfp match  
SFP matched, dTx=180585, dRx=145855, alpha=72169888  
executing: mode slave  
Locking PLL  
executing: calibration  
Found phase transition in EEPROM: 6900ps  
executing: ptp start
```

Now you should have the White Rabbit PTP Core running in WR Slave mode. WRPC needs to make a calibration of t24p phase transition value. It has to be done only once for a new bitstream and is performed automatically when WRPC runs in Slave mode. That is why it is very important, even if WRPC is meant to run in Master mode, to configure it to Slave for a moment and connect it to any WR Master. That has to be repeated every time a new bitstream (gateway) is deployed. Measured value is automatically stored to EEPROM and used later in Master or GrandMaster mode.

The Shell also contains the monitoring function which you can use to check the WR synchronization status:

```
wrc# gui
```

The information is presented in a clear, auto-refreshing screen. To exit from this console mode press <Esc>. Full description about information reported by gui is provided in [Appendix B \[WRPC GUI elements\]](#), page 12.

Note: the *Synchronization status* and *Timing parameters* in *gui* are available only in WR Slave mode. When running as WR Master, you would be able to see only the current date and time, link status, Tx and Rx packet counters, lock and calibration status.

```

WR PTP Core Sync Monitor v 1.0
Esc = exit

TAI Time:                Fri, Jan 2, 1970, 03:12:46

wru1: Link up    (RX: 742, TX: 213), mode: WR Slave  Locked  Calibrated

PTP status: slave

Synchronization status:

Servo state:            TRACK_PHASE
Phase tracking:         ON
Synchronization source:
Aux clock status:

Timing parameters:

Round-trip time (mu):    833913 ps
Master-slave delay:      398192 ps
Master PHY delays:       TX: 10 ps, RX: 163610 ps
Slave PHY delays:        TX: 0 ps, RX: 126000 ps
Total link asymmetry:    37529 ps
Cable rtt delay:         78680 ps
Clock offset:            -2 ps
Phase setpoint:          5341 ps
Skew:                    -2 ps
Manual phase adjustment: 0 ps
Update counter:          184

```

If you want to store the statistics from WRPC operation, it's probably better to use the *stat cont* shell command. It reports the same information as GUI but in a form which is easier to parse and analyze:

```

wrc# stat cont
lnk:1 rx:416 tx:118 lock:1 sv:1 ss:'TRACK_PHASE' aux:0 sec:94197 \
nsec:793068184 mu:836241 dms:400556 dtxm:10 drxm:163610 dtxs:0 drxs:128400 \
asym:35129 crtt:544221 cko:-5 setp:7667 hd:61479 md:37221 ad:65000 ucnt:101 \
temp: 45.6875 C
lnk:1 rx:417 tx:119 lock:1 sv:1 ss:'TRACK_PHASE' aux:0 sec:94198 \
nsec:293076296 mu:836253 dms:400562 dtxm:10 drxm:163610 dtxs:0 drxs:128400 \
asym:35129 crtt:544233 cko:-4 setp:7663 hd:61485 md:37259 ad:65000 ucnt:102 \
temp: 45.6875 C
(...)

```

If you have a DIO Mezzanine board plugged to your SPEC, you can check the synchronization performance by observing the offset between 1-PPS signals from the WR Master and WR Slave. White Rabbit PTP Core generates 1-PPS signal to the LEMO connector No. 1 on DIO Mezzanine. However, please remember to use oscilloscope cables having the same length and type (with the same delay), or take their delay difference into account in your measurements.

4 Troubleshooting

My computer hangs on loading *spec.ko* or *fmc-trivial.ko* driver.

This will occur when you try to load the *spec.ko* kernel driver while your *spec-vuart* is running and trying to get messages from Virtual-UART's registers inside WRPC. Please remember to quit *spec-vuart* before reloading the driver.

I want to synthesize WRPC but *hdlmake* does nothing, just quits without any message.

Please check if you have the Xilinx ISE-related system variables set correctly (*settings32.sh* script provided by Xilinx sets them) and make sure you have overwritten the *\$XILINX* variable to:

```
$ export XILINX=/opt/Xilinx/<version>/ISE_DS
```

or similar, if your installation folder differs from default.

WR PTP Core seems to work but I observe on my oscilloscope that the offset between 1-PPS signals from WR Master and WR Slave is more than 1 ns.

If you're trying to synchronize the SPEC board to WR Switch please remember to read the document and perform the WR Calibration to find out the values of deltaRx and deltaTx for your firmware. Check if the oscilloscope cables you use have the same delays (or take the delay difference into account in your measurements).

5 Questions, reporting bugs

If you have found a bug, you have problems with White Rabbit PTP Core or one of the tools used to build and run it, you can write to our mailing list white-rabbit-dev@ohwr.org

Appendix A WRPC Shell Commands

help reports the available commands in this instance of WRPC

ver	prints which version of wrpc is running
config	prints the Kconfig file used to build this instance of WRPC. It is an optional command, enabled at build time by CONFIG_CMD_CONFIG
verbose <digits>	Set PPSi verbosity. See the PPSi manual about the meaning of the digits (hint: verbose 1111 is a good first bet too see how the PTP system is working)
pll init <mode> <ref_channel> <align_pps>	manually run <code>spll_init()</code> function to initialize SoftPLL
pll cl <channel>	check if SoftPLL is locked for the channel
pll sps <channel> <picoseconds>	set phase shift for the channel
pll gps <channel>	get current and target phase shift for the channel
pll start <channel>	start SoftPLL for the channel
pll stop <channel>	stop SoftPLL for the channel
pll sdac <index> <val>	set the dac
pll gdac <index>	get dac's value
gui	starts GUI WRPC monitor
stat	prints one line log message
stat cont	prints log message for each second (Esc to exit back to shell)
stat bts	prints bitslide value for established WR Link, needed by calibration procedure
ptp start	start WR PTP daemon
ptp stop	stops WR PTP daemon
mode	prints available WR PTP modes
mode gm master slave	sets WRPC to operate as Grandmaster clock (requires external 10MHz and 1-PPS reference), PTP Master or PTP Slave. After setting the mode ptp start must be re-issued
calibration	tries to read t2/4 phase transition value from EEPROM (in WR Master or GrandMaster mode), or executes the t24p calibration procedure and stores its result to EEPROM (in WR Slave mode)
time	prints current time from WRPC

<code>time raw</code>	prints current time in a raw format (seconds, nanoseconds)
<code>time set <sec> <nsec></code>	sets WRPC time
<code>time setsec <sec></code>	sets only seconds part of WRPC time (useful for setting time in GrandMaster mode, when nanoseconds counter is aligned to external 1-PPS and 10 MHz)
<code>time setnsec <nsec></code>	sets only nanoseconds part of WRPC time
<code>sfp detect</code>	prints the ID of currently used SFP transceiver
<code>sfp erase</code>	cleans the SFP database stored in FMC EEPROM
<code>sfp add <ID> <deltaTx> <deltaRx> <alpha></code>	stores calibration parameters for SFP to the database in FMC EEPROM
<code>sfp show</code>	prints all SFP transceivers stored in database
<code>sfp match</code>	tries to get calibration parameters from database for currently used SFP transceiver (<code>sfp detect</code> must be executed before <code>match</code>)
<code>init erase</code>	cleans initialization script in FMC EEPROM
<code>init add <cmd></code>	adds shell command at the end of initialization script
<code>init show</code>	prints all commands from the script stored in EEPROM
<code>init boot</code>	executes the script stored in FMC EEPROM (the same action is done automatically when WRPC starts after resetting LM32)
<code>mac get</code>	prints WRPC's MAC address
<code>mac getp</code>	re-generates MAC address from 1-wire digital thermometer or EEPROM
<code>mac set <mac></code>	sets the MAC address of WRPC
<code>mac setp <mac></code>	sets MAC address to the 1-wire EEPROM (if available)
<code>sdb</code>	prints devices connected to the Wishbone bus inside WRPC
<code>ip get</code>	
<code>ip set <ip></code>	reports or sets the IPv4 address of the WRPC (only available if <code>CONFIG_ETHERBONE</code> is set at build time)
<code>w1w <offset> <byte> [<byte> ...]</code>	
<code>w1r <offset> <len></code>	If <code>CONFIG_W1</code> is set and a OneWire EEPROM exists, write and read data. For writing, <code>byte</code> values are decimal

Appendix B WRPC GUI elements

TAI Time:	current state of device's local clock
Rx: / Tx:	Rx/Tx packets counters
mode:	operation mode of White Rabbit PTP Core - <WR Master, WR Slave>
< Locked, NoLock >	SoftPLL lock state
Servo state:	current state of WR servo state machine - <Uninitialized, SYNC_SEC, SYNC_NSEC, SYNC_PHASE, TRACK_PHASE>
Phase tracking:	is phase tracking enabled when WR Slave is synchronized to WR Master - <ON, OFF>
Synchronization source:	network interface name from which WR daemon gets synchronization - <wru1>
Round-trip time (mu):	round-trip delay in picoseconds ($delay_{MM}$)
Master-slave delay:	estimated one-way (master to slave) link delay ($delay_{MS}$)
Master PHY delays:	transmission/reception delays of WR Master's hardware ($\Delta_{TXM}, \Delta_{RXM}$)
Slave PHY delays:	transmission/reception delays of WR Slave's hardware ($\Delta_{TXS}, \Delta_{RXS}$)
Total link asymmetry:	WR link asymmetry calculated as $delay_{MM} - 2 \cdot delay_{MS}$
Cable rtt delay:	round-trip fiber latency
Clock offset:	Slave to Master offset calculated by PTP daemon ($offset_{MS}$)
Phase setpoint:	current Slave's clock phase shift value
Skew:	the difference between current and previous estimated one-way link delay
Update counter:	the state of counter incremented every time the WR servo is updated